Multi-FPGA Quantum Circuit Simulation: A Distributed Statevector Architecture on Four Xilinx Alveo U55C Accelerators

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Exact statevector simulation of quantum circuits needs memory that doubles with every qubit. We distribute a complex128 statevector over the high bandwidth memory of one, two or four AMD Alveo U55C cards in one host and evaluate the design on the physical cards. A single primitive, a $\mathbf{2 \times 2}$ complex128 matrix applied to amplitude pairs under a per operation slice enable, lets the cards execute every gate that needs no amplitudes of another card, including diagonal, controlled and permutation gates on the partition qubits and wide multi controlled gates, while a compiler packs gates into passes over memory and moves amplitudes only for the remaining gates. Each card runs 8 engines, which use all 32 memory channels of the card. The system simulates 29 qubits on one card, 30 on two and 31 on four cards, the largest widths its memory layout allows; all 166 checked instances agree with an independent reference to double precision rounding. On the same cards the quantum Fourier transform at 20 qubits runs 569 times faster than in a baseline design that computes the gates on partition qubits on the host, whose timing a microbenchmark model reproduces without fitted parameters. Ablations, timing breakdowns, scaling, energy and host measurements show where the time goes: distribution adds capacity, not speed, because exchanges between cards pass through host memory, and Qiskit Aer on the 128 core host is faster for almost all circuits measured.

Keywords: Multi-FPGA, Quantum circuit simulation, Distributed statevector, HBM2, Alveo U55C, Quantum Fourier transform, Complex128 precision

1 Introduction

Classical simulation of quantum circuits remains essential for algorithm development, compiler verification and the validation of quantum hardware [1, 2]. Exact statevector simulation stores all 2^n complex amplitudes of an n qubit register; in complex128 (16 bytes per amplitude) this is 4 GiB at $n = 28$, 16 GiB at $n = 30$ and 64 GiB at $n = 32$, so that every additional qubit doubles both memory and work. Distributing the state over several devices is the established way to go beyond one device, both on CPU clusters and on GPUs [3–8]. On FPGAs, statevector designs have mostly targeted one device [9–14]; the Trefoil system distributes very large state vectors over several FPGAs backed by SATA disk arrays [15, 16].

This paper studies a different point of that design space: a statevector held entirely in the high bandwidth memory of four AMD Alveo U55C cards in one host, with complex128 arithmetic throughout. It is the multi card continuation of the single card complex64 simulator of the companion paper [14], which reaches 30 qubits on one card and contains the comparison of that design with CPU and GPU simulators. We compare two ways of dividing the work between host and cards. In a baseline design, one kernel per card applies the local gates while every gate on a partition qubit is computed on the host after the card's whole slice has been moved over PCIe. Our engine instead executes every gate that does not need another card's amplitudes inside the FPGA, including diagonal and controlled gates on partition qubits, and moves data only for the remaining gates.

The contributions, each supported by measurements on the four cards, are:

1. A complex128 statevector engine for the U55C with one compute unit per group of HBM pseudo channels, built around a single 2×2 multiplexed primitive with a per operation slice enable, and a host runtime that distributes the state over one, two or four cards. The engine holds 29 qubits on one card and 31 qubits on four cards, verified amplitude by amplitude against an independent reference (Section 7.1).
2. Execution of diagonal, partition controlled and partition permutation gates without data movement (F1 to F3), and a hardware specific compiler rewrite for X type gates on partition qubits (F4). Ablations quantify each of them (Section 7.6).
3. A measured account of where the time goes: per phase timing, bytes moved over PCIe, scaling over one, two and four cards, the sensitivity to the host, the board energy, and a model of the baseline design built only from microbenchmarks that explains its timing curve (Sections 7.4 to 7.9).
4. An honest comparison with Qiskit on the same host and the same circuits (Section 7.7).

We make no claim that the FPGA system outperforms CPU or GPU statevector simulators. Section 7.7 shows where Qiskit Aer on the 128 core host is faster and where it is not; the GPU comparison of the same card type is reported in the companion paper [14].

2 Related Work

2.1 Distributed statevector simulation on CPUs and GPUs

Exact statevector simulation beyond the memory of one node distributes the 2^n amplitudes over many processes, each holding a contiguous slice selected by the top qubits, and exchanges amplitudes when a gate acts on such a global qubit. qHiPSTER, later the Intel Quantum Simulator, combined vectorisation, multithreading, cache blocking and overlap of computation with communication and simulated circuits of up to 40 qubits on a supercomputer, limited by memory and network bandwidth [3, 17]. QuEST is a hybrid multithreaded, distributed and GPU accelerated simulator that benchmarked random circuits of up to 38 qubits on up to 2048 nodes [4]. Häner and Steiger used generated and optimised kernels and a scheduling algorithm that reduces communication to simulate a 45 qubit circuit on 8192 nodes [5]; the Jülich simulator reached 48 qubits with an adaptive encoding of the wave function and showed close to ideal weak scaling on several supercomputers [6]; mpiQulacs is a distributed statevector simulator optimised for A64FX clusters [7]. On GPUs, HyQuas selects among partitioning methods per circuit pattern to improve data locality [18], and the cuQuantum SDK provides composable GPU primitives for quantum circuit simulation [8]; Qulacs is a widely used single node CPU and GPU simulator [19]. Simulations at 49 qubits and beyond rely on other representations, such as tensor network contraction and low depth circuit structure [20–24].

Two techniques of these systems are central to the present work. First, a gate that is diagonal in the computational basis never needs amplitudes of another process, even when it acts on a global qubit, and a gate controlled by a global qubit acts only on the processes whose bit is one; second, global and local qubits can be swapped once so that a sequence of gates on a global qubit runs locally (global to local remapping), and gates can be scheduled to reduce the number of such exchanges [5]. Executing diagonal gates on partition qubits without exchange is the first technique; it is not new as an algorithmic idea, and we do not claim it as such. What this paper contributes is the implementation of these techniques inside FPGA kernels: the slice enable lets every card's HBM engines execute partition diagonal, partition controlled and partition permutation gates in their streaming passes without host arithmetic or PCIe traffic, and remaps run as device copies inside a card and as half slice exchanges across cards.

2.2 FPGA statevector simulators

FPGA emulation of quantum circuits was proposed by Khalid, Zilic and Radecka [9]. Lee et al. presented a serial parallel FPGA emulation architecture with the QFT and Grover's search as case studies and a linear reduction of resources compared with pipelined designs [10]; Pilch and Długopolski proposed a universal FPGA emulator with a standard programming interface [11]; Mahmud, El-Araby and Caliga studied scalable reconfigurable emulation at high precision and throughput [12]. More recent work includes Q2Logic, a coarse grained reconfigurable overlay specialised for Schrödinger style simulation [25], PCQ, a parallel pipelined dataflow architecture with data decoupling that partitions computation into non interacting subsets [13], and a

Table 1 Platform. Device values from the AMD U55C data sheet [30]; host values captured by script at measurement time (`results/hardware_manifest.json`).

FPGA device	AMD XCU55C on Alveo U55C (4 cards, shell xilinx_u55c_gen3x16_xdma_base_3)
HBM2 per card	16 GiB in 32 pseudo channels of 512 MiB
Host CPU	2 × AMD EPYC 7742 64-Core Processor, 128 cores, 256 threads
Host memory	503 GiB
Card NUMA nodes	01: 0, 21: 0, 41: 0, a1: 1
OS, XRT, Vitis	Ubuntu 20.04.6 LTS (5.15.0-139-generic), XRT 2.16.204, Vitis 2023.2
Python stack	Python 3.8.20, NumPy 1.24.4, Qiskit 1.2.4, Qiskit Aer 0.15.1

scalable accelerator architecture that addresses parallelism and the routing of state components [26]. These designs target one device. Statevector simulation over several FPGAs has been demonstrated by the Trefoil system, which combines multiple FPGAs with SATA disk arrays (2 PB of storage in eight storage subsystems) to hold state vectors of up to 47 qubits, at the cost of disk bandwidth (an H gate on 30 qubits takes about 10 minutes) [15]; its simulator, Qu-Trefoil, builds on this storage based platform [16]. The present work takes the opposite design point: the whole state stays in the HBM of the cards and moves over PCIe only when a gate requires amplitudes of another card, which limits the width to what the HBM of the cards holds but keeps every gate at HBM bandwidth. The companion paper of this work [14] reaches 30 qubits in complex64 on one U55C with HBM and compares it with CPU and GPU simulators; the present paper is the multi card, complex128 continuation that the companion paper names as future work. Further related designs of the author on the same platform, a native RTL statevector engine in FP64 [27], real time error correction decoders [28] and a matrix product state engine [29], are available as preprints.

3 System Architecture

3.1 Hardware platform

The system is a single host server with four AMD (Xilinx) Alveo U55C accelerator cards on PCIe Gen3 x16 (Table 1). Each card carries the XCU55C device and 16 GiB of HBM2 organised as 32 pseudo channels (PCs) of 512 MiB; every HBM AXI port of a kernel reaches any PC through the HBM switch [30, 31]. Device capacities in Table 1 are those of the data sheet [30]; utilisation, achieved clock and PC bindings are taken from the post route reports of the bitstreams used here (Section 7.3). The host has 128 cores in 2 sockets and 503 GiB of memory; three cards are attached to NUMA node 0 and one to node 1 (Section 7.8).

3.2 Distribution of the statevector

An n qubit state of 2^n complex128 amplitudes (16 bytes each) is split into $S = C\,P$ *slices* of 2^L amplitudes, $L = n - \log_2 S$, where $C \in \{1, 2, 4\}$ is the number of cards and P the number of slices per card. Physical qubits $0, \ldots, L-1$ are local to a slice; the next $\log_2 P$ physical qubits select the slice inside a card and the top $\log_2 C$ select the card.

Card j therefore holds every amplitude whose top $\log_2 C$ index bits equal j, and each card divides its share into P slices, each stored in one HBM buffer (Section 3.4). We call the $\log_2 S$ qubits above L the *slice qubits*; the top $\log_2 C$ of them are the *partition qubits*. Capacities follow directly: an n qubit complex128 state needs 2^{n+4} bytes, i.e. 16 MiB at $n = 20$, 4 GiB at $n = 28$, 8 GiB at $n = 29$ and 16 GiB at $n = 30$, so that four cards (64 GiB in total) can hold up to $n = 31$ when the state uses all PCs (Section 7.2 explains why not $n = 32$).

3.3 Baseline design

The single card design of the companion paper [14] uses sixteen HBM data ports in complex64. When the datapath was widened to complex128 for this work, the sixteen port kernel no longer routed: the implementation failed with routing congestion and no bitstream was produced. The data ports were therefore halved, which gives the baseline kernel. Each data bank needs its own AXI port and the gate sequence one more, so the design could not simply spread the state over all 32 PCs. The baseline design thus uses one compute unit per card with eight HBM data ports and one gate buffer port, bound to HBM[0] to HBM[7] and HBM[8]; amplitude i of a card's slice is stored in port i mod 8, so each card addresses nine PCs (4.5 GiB) of its 16 GiB. The kernel implements the 19 opcodes of Table 2 in complex128 arithmetic; gate angles are transferred as float32 and widened in the kernel. With $n_\ell = n - \log_2 C$ local qubits, a single qubit gate on local qubit q visits the $2^{n_\ell - 1}$ amplitude pairs

$$i_0 = \big((k \gg q) \ll (q+1)\big) \,|\, \big(k \,\&\, (2^q - 1)\big), \qquad i_1 = i_0 + 2^q, \qquad k = 0, \ldots, 2^{n_\ell - 1} - 1, \quad (1)$$

and applies the 2×2 gate matrix to $(\alpha_{i_0}, \alpha_{i_1})$. CX with control c and target t swaps α_i and $\alpha_{i \oplus 2^t}$ for every i with $(i \,\&\, 2^c) \neq 0$ and $(i \,\&\, 2^t) = 0$; CZ negates

$$\alpha_i \leftarrow -\alpha_i \quad \text{for every } i \text{ with } (i \,\&\, 2^c) \neq 0 \text{ and } (i \,\&\, 2^t) \neq 0. \quad (2)$$

A controlled phase $\mathrm{CP}(\theta)$ whose control is partition bit p_b of card index j and whose target is local qubit q_t changes only amplitudes of the card itself:

$$\alpha'_i = \begin{cases} e^{i\theta}\, \alpha_i & \text{if } \big((j \gg p_b) \,\&\, 1\big) = 1 \text{ and } \big((i \gg q_t) \,\&\, 1\big) = 1, \\ \alpha_i & \text{otherwise.} \end{cases} \quad (3)$$

A host process per card drives the kernel and takes part in a three barrier exchange through shared host memory (Section 4). Its three routing paths are executed as follows:

- *Local gates* (all qubits below the partition qubits) are lowered to the 19 opcodes and executed by the kernel on every card.
- *Diagonal gates on partition qubits* are executed on the host: each card reads its slice from HBM over PCIe, multiplies it by the card dependent phase in NumPy and writes it back. No amplitudes cross between cards, but each such gate moves the whole slice over PCIe twice.

Table 2 Opcodes of the baseline kernel (19, complex128 arithmetic), as defined in its source.

Opcode	Gate	Opcode	Gate	Opcode	Gate	Opcode	Gate
0	I	5	S	10	$\sqrt{X}^\dagger$	15	$U_2(\phi, \lambda)$
1	H	6	T	11	$R_X(\theta)$	16	$U_3(\theta, \phi, \lambda)$
2	X	7	$S^\dagger$	12	$R_Y(\theta)$	17	CX
3	Y	8	$T^\dagger$	13	$R_Z(\theta)$	18	CZ
4	Z	9	$\sqrt{X}$	14	$P(\phi)$		

- *Other gates on partition qubits* are executed on the host after the three barrier exchange: each card reads its slice, publishes it to shared memory, computes its new slice from its partner slices in NumPy and writes it back.

Hence the baseline design performs gate arithmetic on the host for every gate that touches a partition qubit.

3.4 Engine

The engine keeps the partition by the top qubits and the three gate classes of the baseline design, and moves every gate that does not require amplitudes of another slice onto the FPGA. It reuses the MUX2 pass engine of the companion single card simulator [14], widened to complex128 and extended by a per operation slice enable.

One primitive.

Every gate is compiled into MUX2 records. A record (t, c, M_0, M_1) applies the 2×2 complex128 matrix M_1 to the amplitude pairs $(i_0, i_0 \mid 2^t)$ whose index has all bits of the mask c set, and M_0 to the others. A single qubit gate is $M_0 = M_1 = U$; a controlled U is $M_0 = I$, $M_1 = U$ with the controls in c; a two qubit diagonal gate with diagonal (d_0, d_1, d_2, d_3) on qubits (a, b) is one record on b with $c = 2^a$, $M_0 = \mathrm{diag}(d_0, d_2)$, $M_1 = \mathrm{diag}(d_1, d_3)$; any other two qubit gate takes three records (cosine sine decomposition). Multi controlled gates (Toffoli, the Grover oracle) are a single record. Matrices are stored as float64, so no gate parameter is rounded below double precision.

Slice enable.

Each record carries $(s_{\mathrm{mask}}, s_{\mathrm{val}}, s_{\mathrm{inv}})$; the engine that holds the slice with logical index s executes the record only if $\big((s \,\&\, s_{\mathrm{mask}}) = s_{\mathrm{val}}\big) \oplus s_{\mathrm{inv}}$. The slice index is a kernel argument of every launch. This turns three classes of gates on slice qubits into local work:

F1 *Diagonal gates on slice qubits* (Z, S, T, R_Z, P, CZ, CP, CR_Z, R_{ZZ} and any diagonal gate) become scalar phase records under a slice enable; they move no data at all and run inside a pass that already streams the slice.

F2 *Gates controlled by slice qubits* with a local target (for example CX from a partition qubit to a local qubit) run as ordinary records on the slices whose control bits are one.

F3 *X, CX and Toffoli acting only on slice qubits* permute whole slices; they are implemented by relabelling which buffer holds which logical slice, again without data movement. SWAP gates are always absorbed into the logical to physical qubit map.

Only non diagonal gates whose target is a slice qubit need amplitudes of other slices. For these the compiler swaps the slice qubit with a local qubit (global to local remapping, as in distributed CPU and GPU simulators [5–7]) and keeps executing locally until the local qubit is needed again. A remap exchanges half of each slice with its partner slice: between slices of the same card it runs as a device copy kernel inside HBM; between cards the moving halves cross PCIe through host memory. Each of F1 to F3 can be switched off, in which case the gate falls back to the remap path; Section 7.6 uses these switches for the ablations.

Reach of every pseudo channel.

The engine avoids both limits of the baseline kernel. Each compute unit has a single AXI port, and its instruction segment is stored in the same buffer as its slice, so no port is spent on instructions; and the compute units, not the ports of one unit, provide the parallelism. The main build has 8 complex128 engines per card, each bound to 4 PCs, so that the 32 PCs of each card hold the state (Section 7.3); with the four exchange units this uses 16 of the HBM port connections of the card. A build with sixteen complex128 engines (two PCs each) placed but failed routing with conflicted nets inside the engines, the same congestion that limited the baseline kernel; eight engines with four PCs each route at 300 MHz and still reach every PC, which is what matters for capacity. Engine count and memory reach are thus decoupled.

Pass engine.

Each compute unit (8 per card in the main build, each bound to 4 PCs) holds one slice while a slice fits one PC ($P = 8$), and for the widest states one slice per PC, which it processes in turn ($P = 32$). It streams each slice through on chip tiles of 2^k amplitudes ($k \leq 11$). A pass loads every tile once, applies all records of the pass whose qubits lie in the tile window (up to four high qubits gathered per tile), and writes the tile back, so a sequence of gates costs one read and one write of the slice instead of one per gate. Inside the pass the compute stage updates two amplitude pairs per cycle at an initiation interval of one; the 512 bit HBM ports move four amplitudes per cycle in each direction. All cards execute the same instruction segment concurrently; the host starts every compute unit of every card and waits for all of them.

4 Host Runtime and Distributed Protocol

4.1 Baseline design: three barrier exchange

In the baseline design one host process per card drives the card's kernel; the processes share three `multiprocessing.Barrier` objects B_A, B_B, B_C and a read buffer $R[j]$

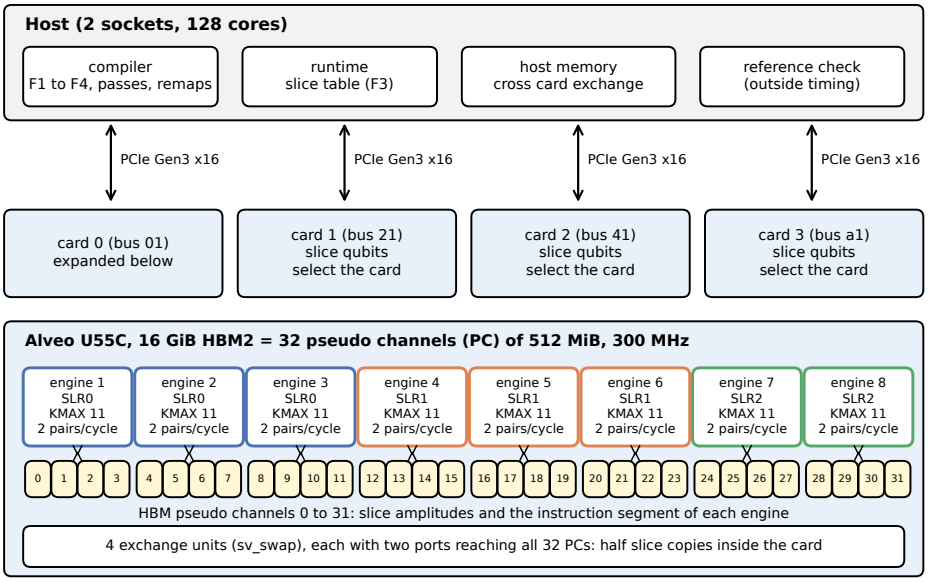


Fig. 1 Data flow of the engine on four cards. Local, slice controlled (F2) and slice diagonal (F1) gates run in the HBM passes of every engine; slice relabelling (F3) changes only the slice table on the host; remaps exchange half slices inside a card (device copy) or across cards (PCIe and host memory).

and write buffer $W[j]$ per card in shared host memory. A gate on partition qubits that is not diagonal is executed in three phases: each card reads its slice from HBM into $R[j]$ and waits at B_A; each card computes its new slice from the slices of its partner cards in NumPy, writes it to $W[j]$ and waits at B_B; each card writes $W[j]$ back to HBM and waits at B_C. Algorithm 1 shows the Hadamard gate on partition qubit q_p. The card whose partition bit is one holds the amplitudes with $b_{q_p} = 1$, so its new amplitudes are $(R[j'] - \alpha^{(j)})/\sqrt{2}$, with the partner's amplitude first.

Every gate that touches a partition qubit therefore moves the whole slice of every card over PCIe twice: the exchange path for the exchange, and the diagonal path because the phase is multiplied on the host. Table 10 reports the measured bytes per gate class.

4.2 Engine runtime

The engine runtime drives all engines of all cards from one host process. For each segment it uploads the instruction stream to every slice buffer, starts every engine of every card with its logical slice index, and waits for all of them; cards run concurrently. A slice relabelling (F3) only updates the table that maps buffers to logical slices. A remap swaps half slices: when both partner slices are on the same card and the moving chunks are large, the card's exchange kernel copies them inside HBM; otherwise the moving halves are read over PCIe into host memory, swapped there and written back,

Algorithm 1 Three barrier exchange for H on partition qubit q_p (card j, baseline design).

Require: partition bit index $p_b = q_p - n_\ell$, local slice $\alpha^{(j)}$

1: $R[j] \leftarrow$ slice read from HBM ▷ 2^{n_ℓ} amplitudes over PCIe
2: wait at B_A
3: $b \leftarrow (j \gg p_b)\ \&\ 1; \quad j' \leftarrow j \oplus (1 \ll p_b)$
4: **if** $b = 0$ **then**
5: $\quad W[j] \leftarrow (\alpha^{(j)} + R[j'])/\sqrt{2}$
6: **else**
7: $\quad W[j] \leftarrow (R[j'] - \alpha^{(j)})/\sqrt{2}$
8: **end if**
9: wait at B_B
10: write $W[j]$ to HBM ▷ 2^{n_ℓ} amplitudes over PCIe
11: wait at B_C

with one transfer per buffer when the chunks are small. The runtime records the time and the bytes of every phase (Table 3).

4.3 Compilation flow

Every benchmark circuit is built once by `bench/circuits.py` (fixed seeds) and passed unchanged to the FPGA engines and to the Qiskit CPU runs of Section 7.7; QASM copies are part of the reproduction package. The flow from a Qiskit circuit to engine records has five steps, all on the host and all included in T_{compile} (Table 3):

1. *Gate intake.* Gates of one or two qubits are taken as unitary matrices from Qiskit (`Operator`); multi controlled X, Z and phase gates are taken directly as one controlled 2×2 operation. Such a gate stays one record as long as its target and its local controls fit the tile window and its other controls sit on slice qubits, where the slice enable (F2) applies them; with k tile qubits and s slice qubits this allows gates of up to $k + s$ qubits. Wider gates, every gate wider than the tile window when F2 is switched off, and gates without a controlled form are decomposed by the Qiskit transpiler with `basis_gates=[u, cx]`, `optimization_level=1`. No other transpilation is applied, so the FPGA and CPU runs see the same gate sequence.
2. *Fusion and relabelling.* Runs of single qubit gates on one qubit are multiplied into one matrix; SWAP gates are absorbed into the logical to physical qubit map and cost nothing.
3. *Classification.* With the current qubit map each operation is classified as local, slice controlled (F2), slice diagonal (F1), slice permutation (F3) or exchange (Section 3.4).
4. *Scheduling.* Operations are appended to the current pass while their qubits fit the tile window; an exchange operation first triggers a remap that brings its slice qubit to a local position, choosing as victims the local qubits used furthest in the future, and the remapped qubit stays local for all later gates on it. A multi controlled gate wider than the tile window is placed by one remap that swaps its target with one

of its own controls, which then acts through the slice enable, and moves surplus local controls to free slice positions; this keeps the Grover oracle a single record up to $n = 14$ on one and two cards and $n = 16$ on four cards.

5. *Encoding.* Passes between remaps form one segment, uploaded once to every engine and executed by one launch per engine.

In a statevector simulator the cost is set by the number of full sweeps over the state and by the amplitudes that cross slices, not by circuit depth. GHZ preparation, a serial chain of CX gates, shows what this means for the compilation. On the engine all CX gates of the textbook GHZ chain whose qubits are local share one pass, so depth does not cost sweeps; what costs is each CX whose target is a slice qubit, because the target must first be brought into the slice (one remap per slice qubit, 5 remaps with four cards and 8 engines per card). A logarithmic depth construction [32] reduces depth but leaves this count almost unchanged (5). The compiler therefore applies a hardware specific rewrite (F4): for an X type gate (CX, Toffoli) whose target is a slice qubit and whose controls are local, it moves the controls up into slice positions instead of moving the target down. The remap costs the same, but the gate and every later CX among these qubits then become slice relabelling (F3) or slice controlled (F2) operations; the GHZ chain needs 1 remap at every width. Because the rewrite can also increase traffic (it does for the controlled modular multipliers of Shor's circuit), the compiler schedules every circuit with and without F4 and keeps the schedule with fewer remaps. Section 7.6 reports the measured times of all variants.

5 Benchmark Circuits

All circuits are produced by one generator with fixed seeds; their gate counts and routing are generated from the compiler counters and reported in Table 9.

Quantum Fourier transform.

The n qubit QFT consists of n Hadamard gates, $n(n-1)/2$ controlled phase gates $\mathrm{CP}(\pi/2^{j-i})$ and $\lfloor n/2 \rfloor$ final SWAP gates, a total of $n(n+1)/2 + \lfloor n/2 \rfloor$ gates [33]. In the baseline design with the two top qubits as partition qubits, the routing of the QFT at $n = 12$ is 59 local, 21 diagonal and 4 exchange gates (the two Hadamard gates on the partition qubits and the two SWAP gates that cross the partition boundary), counted gate by gate on the circuit before lowering. On the engine SWAP gates are relabellings and the counts of Table 9 apply. The QFT is also run on a Haar random input state and as a round trip (a random product state, QFT, inverse QFT), whose output equals its input.

GHZ and Bell states.

The GHZ state is prepared by the textbook chain (H on qubit 0 and $\mathrm{CX}(i, i+1)$), by a fan out from the top qubit, and by the logarithmic depth construction of Cruz et al. [32]; the Bell state is H on qubit 0 followed by CX from qubit 0 to qubit $n-1$.

Grover search.

For one marked state among $N = 2^n$ and $\theta = \arcsin(2^{-n/2})$, k iterations give the success probability $P(k) = \sin^2\big((2k+1)\theta\big)$. We use the standard iteration count $k_{\text{opt}} = \lfloor \pi/(4\theta) \rfloor$, which lies next to the first maximum of $P(k)$ and guarantees $P(k_{\text{opt}}) \geq 1 - 2^{-n}$ [34, 35]; because $P(k)$ is periodic in k, larger iteration counts can come closer to one, at several times the cost. All Grover runs use k_{opt}, and we report the measured target probability against $P(k_{\text{opt}})$; for $n = 4$ this probability is 0.9613. The oracle and diffuser use one multi controlled X gate each, which the engine executes as a single record.

Phase estimation (Shor).

The order finding component of Shor's algorithm [36] is run as quantum phase estimation of the modular multiplication by $a = 7$ modulo $N = 15$ (order $r = 4$) with a counting register of t qubits and four work qubits; the counting register must show four peaks of probability $1/4$ at multiples of $2^t/4$. To scale the phase estimation to the full width we also run it on the phase gate $P(2\pi\phi)$ with $\phi = 5/16$, whose t qubit counting register must return $2^t\phi$ with probability one. A larger order finding instance (for example $N = 21$) needs modular multipliers that we did not implement; the phase estimation on the phase gate covers the same circuit structure at every width.

Random and worst case circuits.

Random circuits have layers of random U3 gates on every qubit and CX or CZ gates on random disjoint pairs. The worst case for the distributed design applies H and RX gates to the top qubits in every layer, so that every layer contains non diagonal gates on partition qubits.

6 Methodology

6.1 What the reported times contain

Every phase is timed on the host with `time.perf_counter` and recorded per run; Table 3 lists the phases and states which of them each reported time includes. Device open and bitstream load (T_{setup}) happen once per session and are reported separately. T_{e2e} is the time from the Qiskit circuit object to the sampled measurement results (1024 shots) and includes compilation, all transfers, all kernel time, all exchanges, the final read back and sampling; it excludes T_{setup} and the reference computation used for fidelity. For the baseline design the same boundaries are used; its worker processes are started once per session.

6.2 Protocol

Each configuration runs one warm up repetition, which is discarded, followed by 1 to 11 timed repetitions (more for short runs, fewer for the widest runs, whose length is bounded by the thermal limit of Section 9). We report the median and the interquartile range. Runs were made with no other job on the host (no build, no other benchmark;

Table 3 Timing phases and their inclusion in the reported times.

Phase	Contents	in T_{e2e}	in T_{kernel}
T_{setup}	device open, bitstream load, worker start (once per session)	no	no
T_{compile}	Qiskit circuit to engine records (both F4 schedules)	yes	no
T_{h2d}	initial state on the device, instruction uploads	yes	no
T_{kernel}	all engine launches, start to completion of the last engine	yes	yes
T_{exch}	remaps: device copies, host exchanges (PCIe and host copy), relabelling	yes	no
T_{d2h}	final read back of all slices	yes	no
T_{post}	sampling of 1024 shots from the slices	yes	no
reference	Qiskit or Aer reference state and error metrics	no	no

checked and logged before every campaign). The CPU frequency governor was ondemand and is recorded with every campaign. All circuits come from one generator with fixed seeds (Section 4.3); random input states are drawn from the Haar measure with a fixed seed per width.

6.3 Correctness metrics

For every reported instance the final state ψ (read back from HBM, outside the timed region) is compared with an independent reference ψ_{ref}: the Qiskit `Statevector` class for $n \leq 20$ and Qiskit Aer in double precision with gate fusion disabled above. We report the fidelity $F = |\langle\psi_{\mathrm{ref}}|\psi\rangle|^2$, the infidelity computed without cancellation as $1 - F = d^2 - d^4/4$ with $d = \min_\phi \|\psi - e^{i\phi}\psi_{\mathrm{ref}}\|$ for normalised states, the largest amplitude error after optimal global phase alignment, and the ℓ_2 error d. A global phase is physically irrelevant and is removed by the alignment; it is nonetheless reported, since the baseline design's gate decompositions drop global phases. Grover runs additionally report the measured target probability against the analytic value $\sin^2((2k+1)\theta)$, and the phase estimation runs the full distribution of the counting register.

6.4 CPU reference timing

To compare the FPGA simulation time with the Qiskit simulation time, on the same host and for exactly the same circuit objects we time (i) the Qiskit `Statevector` class (NumPy, complex128, one thread) and (ii) Qiskit Aer, statevector method, double precision, with its default thread count and with one thread (versions in Table 1). For Aer the circuit is transpiled to the simulator's native gate set with `optimization_level=0`, which keeps multi controlled gates such as the Grover oracle as single gates, and Aer's default gate fusion is left enabled, as a user would run it; the reference states for correctness are computed with fusion disabled. The FPGA engine receives the same circuit object and compiles a multi controlled gate into one record (Section 4.3); the baseline design lowers it to its 19 opcodes with the decomposition of its original code. Only the simulation call is timed; circuit construction and transpilation are excluded for both CPU and FPGA, and the FPGA time we compare against is T_{e2e}, which does include the FPGA side compilation. A comparison with GPU simulators is not repeated here: a detailed FPGA, CPU and CUDA-Q GPU comparison

on the same type of card is the subject of the companion single card study [14], and this paper makes no speed claim against GPU simulators.

6.5 Energy

Board power of each card is read from its management controller (sum of the 12 V PCIe, 12 V auxiliary and 3.3 V rails, the quantity reported by `xbutil`) every 0.1 s during a dedicated energy stage, in which every configuration is repeated several times. The controller refreshes its readings about once per second. For every run the board power of a card is the time weighted mean of the readings in force during the run, including the reading taken before the run started, and the energy is this power times the end to end time, summed over the cards in use; we report the median over the repetitions together with the number of new readings that fell inside the shortest run (Table 19). Host CPU and memory power are not measured, because the RAPL energy counters of this host are readable only with administrator rights, so the energy figures cover the accelerator boards only.

7 Results

7.1 Correctness

Table 4 summarises the comparison of every reported instance with the independent reference (Section 6.3). All 166 checked instances, from $n = 4$ to $n = 31$ on one, two and four cards, 43 of them from Haar random input states, agree with the reference to double precision rounding: the largest amplitude error is 2.00×10^{-13} and the largest infidelity $1 - F$ is 8.07×10^{-25}. In 29 instances the state is bit identical to the reference. The engine keeps every gate matrix in float64; the baseline design transfers gate angles as float32 and widens them in the kernel. Its largest amplitude error is 1.11×10^{-16} for QFT, GHZ and Bell states from $|0 \ldots 0\rangle$, where no rotation angle affects the result (in the QFT every controlled phase acts while its control qubit is still $|0\rangle$), but 1.38×10^{-7} for random circuits, Grover search and Haar random inputs, whose rotations change the state. Grover search with k_{opt} iterations (Table 5) reaches the analytic target probability $\sin^2((2k_{\mathrm{opt}} + 1)\theta)$ to within 1.31×10^{-12}. The counting registers of the phase estimation runs (order finding for $N = 15$ and phase estimation of $P(2\pi\phi)$, rows shor15 and qpe of Table 4) reproduce the probability distribution of the reference to within 8.88×10^{-15} in every outcome.

7.2 Largest simulated width

The capacity follows from the memory layout. Every HBM buffer of a slice lies in one pseudo channel of 512 MiB and holds the slice together with the engine's instruction segment of 4 MiB. For the widest states each card has 32 slices, one per PC, of at most 2^{24} amplitudes (256 MiB), so one card holds $n = 29$, two cards $n = 30$ and four cards $n = 31$. The verified widths reach this capacity on every configuration: 29, 30 and 31 qubits (Table 6). The state then takes 8 GiB of the 16 GiB of each card, spread over all 32 pseudo channels. The next power of two, 2^{25} amplitudes, would fill a pseudo channel completely and leave no room for the instruction segment that shares

Table 4 Correctness of the engine: worst values over all instances of each circuit family, after optimal global phase alignment. Instances: every width and card count of the campaign; zero input $|0\ldots0\rangle$ or Haar random.

Circuit	Input	n	Instances	max $1-F$	max $\lvert\Delta\alpha\rvert$	max ℓ_2
bell	zero	4 to 22	10	1.23×10^{-32}	1.11×10^{-16}	1.11×10^{-16}
ghz	haar	8 to 28	11	3.88×10^{-32}	1.40×10^{-17}	1.97×10^{-16}
ghz	zero	4 to 31	28	1.23×10^{-32}	1.11×10^{-16}	1.11×10^{-16}
grover	zero	4 to 16	7	8.07×10^{-25}	1.04×10^{-14}	8.98×10^{-13}
partition heavy	haar	8 to 28	11	8.50×10^{-31}	7.93×10^{-17}	9.22×10^{-16}
partition heavy	zero	4 to 22	10	4.19×10^{-31}	3.72×10^{-16}	6.47×10^{-16}
qft	haar	8 to 28	11	1.08×10^{-30}	6.40×10^{-17}	1.04×10^{-15}
qft	zero	4 to 31	24	4.57×10^{-30}	8.67×10^{-19}	2.14×10^{-15}
qft iqft	zero	4 to 28	13	2.57×10^{-30}	3.33×10^{-16}	1.60×10^{-15}
qpe	zero	12 to 28	10	1.31×10^{-25}	2.00×10^{-13}	3.62×10^{-13}
rand10	haar	12 to 28	10	3.83×10^{-30}	6.14×10^{-17}	1.96×10^{-15}
rand10	zero	4 to 28	13	3.67×10^{-30}	2.39×10^{-16}	1.92×10^{-15}
shor15	zero	12 to 24	8	5.84×10^{-30}	1.16×10^{-15}	2.42×10^{-15}

Table 5 Grover search with k_{opt} iterations on four cards: measured target probability against $P(k_{\mathrm{opt}}) = \sin^2((2k_{\mathrm{opt}}+1)\theta)$.

n	k_{opt}	$P(k)$ analytic	P measured	difference
4	3	0.961319	0.961319	7.66×10^{-15}
6	6	0.996586	0.996586	1.87×10^{-14}
8	12	0.999947	0.999947	4.55×10^{-14}
10	25	0.999461	0.999461	1.11×10^{-13}
12	50	0.999945	0.999945	2.55×10^{-13}
14	100	1.000000	1.000000	5.80×10^{-13}
16	201	0.999988	0.999988	1.31×10^{-12}

the buffer, so the engine reaches every pseudo channel but fills only half of the HBM (Section 9). At $n = 31$ GHZ preparation and the QFT run on the four cards in 13.7 s and 60.8 s and agree with the reference computed by Qiskit Aer. Earlier attempts at this width were stopped by the thermal guard of one card (Table 7).

7.3 FPGA resources

Table 8 gives the post route utilisation of both bitstreams. The baseline kernel occupies 16.1 % of the LUTs, 8.1 % of the DSPs and 11.8 % of the BRAM and closes timing at 258 MHz. The engine build with 8 compute units uses 37.3 % of the LUTs, 29.8 % of the DSPs, 40.1 % of the BRAM tiles and 6.7 % of the URAM at 300 MHz. The DSPs implement the double precision arithmetic of the 2×2 primitive. The build with sixteen compute units placed but failed in routing (Section 3.4), so the engine count is limited by routing congestion rather than by the number of LUTs or DSPs. The PC bindings of both builds are given in Sections 3.3 and 3.4.

Table 6 Largest width verified against the reference per number of cards and circuit, HBM occupied per card by the state, end to end time (median of the timed repetitions; n/a for the single validation runs) and the largest amplitude error.

Cards	Circuit	largest n	HBM per card (GiB)	T_{e2e} (s)	max $\lvert\Delta\alpha\rvert$
1	ghz	29	8	n/a	1.11×10^{-16}
1	qft	29	8	n/a	1.02×10^{-19}
1	rand10	28	4	21.9	6.10×10^{-19}
2	ghz	30	8	n/a	1.11×10^{-16}
2	qft	30	8	n/a	7.12×10^{-20}
2	rand10	28	2	n/a	5.22×10^{-19}
4	ghz	31	8	13.7	1.11×10^{-16}
4	qft	31	8	60.8	5.42×10^{-20}
4	qft iqft	28	1	20.8	7.38×10^{-17}
4	rand10	29	2	31.1	4.16×10^{-19}

Table 7 Instances stopped by the thermal guard (a card reached 90 °C FPGA or 85 °C HBM): setting of the slice paths, completed repetitions, the card and its temperatures at the stop. The paths column gives F1, F2, F3 and F4 in this order (1 on, 0 off, a: F4 chosen by the compiler, Section 4.3).

Cards	Circuit	n	paths	repetitions	card	FPGA / HBM (C)
4	qft	28	0000	4 of 6	21	90 / 85
4	rand10	8	111a	1 of 3	21	93 / 88
4	grover	16	111a	5 of 6	21	90 / 85
4	qft iqft	24	111a	0 of 6	21	90 / 85
4	qft	31	111a	2 of 2	21	91 / 86
4	qft	30	111a	3 of 4	21	90 / 85
4	ghz	31	111a	0 of 1	21	90 / 85

7.4 Where the time goes

HBM streaming and compute.

A pass reads and writes every slice once. With slices of 2^{24} amplitudes the 8 engines of each card stream 95 GB/s per card, 379 GB/s on four cards (Table 11); this is the measured rate of the design, well below the 460 GB/s peak of the HBM [30], and small slices are dominated by the launch time of 0.8 ms. When a pass carries more operations the engine becomes compute bound: each additional operation at $L = 22$ costs 3.78 ms, close to the 3.50 ms that follow from two amplitude pairs per cycle at 300 MHz.

Engine.

Table 9 shows where the gates go. Of the 420 gates of the QFT at $n = 28$, 268 run locally, 85 under the slice enable (F2) and 44 as slice diagonal records (F1); the SWAP gates are absorbed into the qubit map, and only 9 gates need a remap. The records

Table 8 Post route utilisation (full device including the shell) and achieved kernel clock of the bitstreams used in this paper.

Build	MHz	LUT (%)	FF (%)	DSP (%)	BRAM tiles (%)	URAM (%)
baseline (8 data ports)	258	209,980 (16.1)	250,174 (9.6)	727 (8.1)	238 (11.8)	0 (0.0)
engine, 8 compute units	300	486,506 (37.3)	684,497 (26.2)	2,692 (29.8)	808 (40.1)	64 (6.7)
engine, 16 compute units	placed; routing failed (conflicted nets)					
device total (XCU55C)		1,303,680	2,607,360	9,024	2,016	960

Table 9 Routing of the engine on four cards (compiler counters). "gates" counts the gates of the circuit; the following columns count operations after fusion of consecutive single qubit gates and absorption of SWAP gates into the qubit map, so they do not add up to the gate count: local operations, partition controlled (F2), partition diagonal (F1), slice relabelling (F3) and operations that required a remap; then remaps and HBM passes.

Circuit	n	gates	local	F2	F1	F3	exchange	remaps	passes
bell	20	2	1	0	0	0	1	1	2
ghz	20	20	15	1	0	3	1	1	4
ghz	28	28	23	1	0	3	1	1	6
partition heavy	20	40	15	0	0	0	2	2	2
qft	20	220	116	55	31	0	8	8	24
qft	28	420	268	85	44	0	9	9	76
qft iqft	20	460	258	153	6	0	21	21	59
qft iqft	28	868	563	244	8	0	23	23	169
rand10	20	300	279	0	0	0	21	21	29
rand10	28	420	400	0	0	0	20	20	46

are packed into 76 HBM passes, so the state is swept far fewer times than there are gates. On four cards the FPGA kernels then take 41 % of the end to end time and PCIe transfers (state upload, exchanges across cards, read back) 55 % (Fig. 2); the remaining time is compilation and host bookkeeping. The exchanges are thus the part that more cards make larger (Section 7.5).

Baseline design.

Table 10 gives the same breakdown for the baseline design. For small circuits most of its time is synchronisation of the card processes: 54 % for the QFT at $n = 8$, against 12 % in the kernel. From $n \approx 14$ on, its per gate kernel sweeps dominate, with 98 % of the time at $n = 22$, which is the behaviour modelled in Section 7.9. A circuit whose gates all act on partition qubits never reaches the kernel: the partition heavy circuit at $n = 22$ spends 48 % in PCIe transfers, 26 % in host arithmetic and 27 % in synchronisation. These are the two costs that the engine removes: per gate sweeps by packing records into passes, and host arithmetic by executing partition gates on the cards.

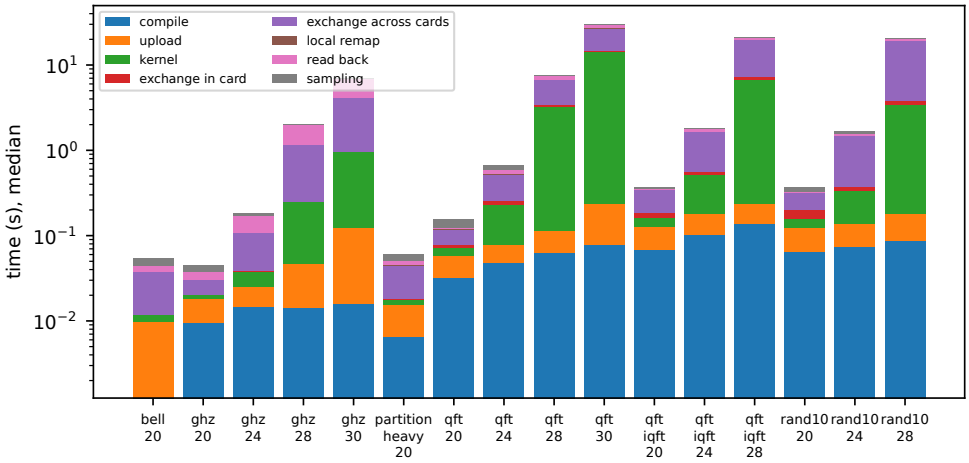


Fig. 2 Per phase time of the engine on four cards (median of the timed repetitions).

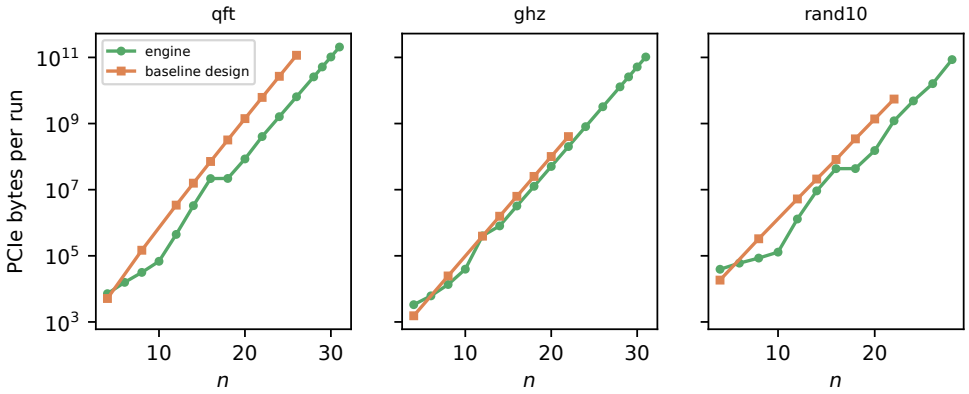


Fig. 3 PCIe bytes per run on four cards, including the state upload and read back, for the baseline design and the engine (the engine also counts its instruction uploads).

7.5 Scaling over one, two and four cards

Distribution over more cards buys capacity, not speed (Table 12, Fig. 4). In 21 of 32 comparisons the two or four card run is slower than the one card run of the same circuit. The best case is the QFT at $n = 28$, with a speedup of 1.62 on two and 1.77 on four cards. The reason is the exchange path. On one card all slices share one HBM, and remaps run as device copies, except for large remaps whose device copy chunks would be too small, which the runtime moves through host memory. With more cards the top qubits become card qubits, and every remap that involves one of them crosses PCIe through host memory, because peer to peer transfers between the cards are not

Table 10 Baseline design on four cards: routing, bytes moved over PCIe and share of the run time per phase (slowest card).

Circuit	n	local	diag.	exch.	PCIe (MiB)	kernel %	PCIe %	host %	sync %
bell	8	1	0	1	0.0	6	32	3	58
bell	14	1	0	1	1.0	19	26	5	50
bell	22	1	0	1	256.0	52	17	17	13
ghz	8	6	0	2	0.0	7	26	5	62
ghz	14	12	0	2	1.5	39	5	25	31
ghz	22	20	0	2	384.0	94	2	2	2
partition heavy	8	0	0	40	0.3	0	31	5	64
partition heavy	14	0	0	40	20.5	0	35	7	58
partition heavy	22	0	0	40	5248.0	0	48	26	27
qft	8	23	13	4	0.1	12	32	3	54
qft	14	83	25	4	15.0	88	5	1	7
qft	22	219	41	4	5888.0	98	1	0	1
qft	26	311	49	4	110592.0	98	0	0	1
qft iqft	8	52	26	10	0.3	11	31	4	54
qft iqft	14	178	50	10	30.5	89	5	1	6
qft iqft	22	458	82	10	11904.0	98	1	0	1
rand10	8	81	10	29	0.3	8	26	4	61
rand10	14	171	11	28	20.0	64	9	7	19
rand10	22	290	14	26	5248.0	94	2	1	2

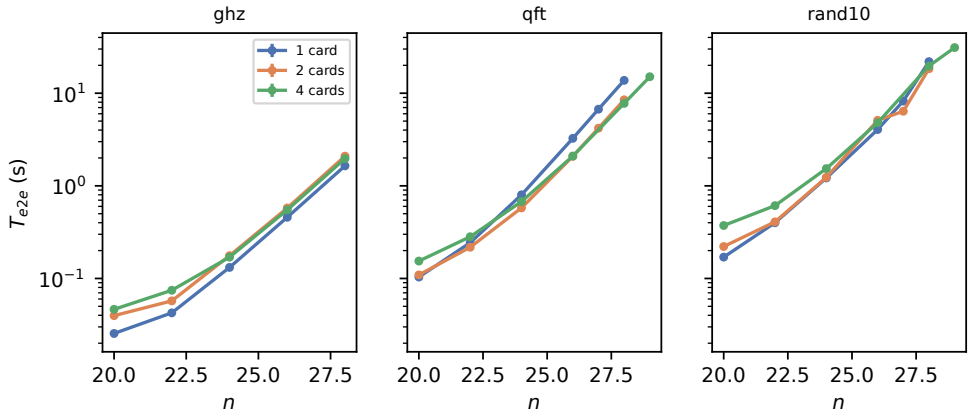


Fig. 4 End to end time on one, two and four cards (median, whiskers: interquartile range).

available on this host (Section 9). For the QFT at $n = 28$ the exchanges take 6 % of the end to end time on one card and 47 % on four cards, of which 3.45 s through the host; for the random circuit they grow from 37 % to 78 %. The more cards hold the state, the more of the run is PCIe traffic; with the present exchange path the four card system is the way to simulate $n = 30$ and $n = 31$, not a way to simulate $n \leq 29$ faster than one card. With the same number of amplitudes per card (weak scaling, n

Table 11 Engine microbenchmarks on four cards, 8 engines per card: launch of an empty segment, one pass with one operation, and the HBM rate it implies (one read and one write of every slice), for slice widths L.

L	slice (MiB)	launch (ms)	one pass (ms)	HBM, all cards (GB/s)	per card (GB/s)
11	0.03125	0.79	0.78	3	1
12	0.0625	0.79	0.781	5	1
13	0.125	0.78	0.52	16	4
14	0.25	1.06	0.593	28	7
15	0.5	0.79	0.609	55	14
16	1	0.77	0.689	97	24
17	2	1.01	0.922	146	36
18	4	0.77	1.21	221	55
19	8	0.77	2.19	245	61
20	16	0.79	3.61	298	74
21	32	0.76	6.55	328	82
22	64	0.83	12.2	353	88
23	128	0.82	23.2	370	92
24	256	1.00	45.3	379	95

Table 12 End to end time T (median), speedup $S = T_1/T_C$ and parallel efficiency $E = S/C$ for C cards.

		1 card	2 cards			4 cards		
Circuit	n	T (s)	T (s)	S	E (%)	T (s)	S	E (%)
ghz	20	0.0255	0.0397	0.64	32	0.0464	0.55	14
ghz	22	0.0425	0.0573	0.74	37	0.0746	0.57	14
ghz	24	0.132	0.176	0.75	37	0.17	0.77	19
ghz	26	0.458	0.576	0.79	40	0.55	0.83	21
ghz	28	1.64	2.09	0.79	39	1.97	0.84	21
qft	20	0.104	0.109	0.95	47	0.154	0.67	17
qft	22	0.243	0.217	1.12	56	0.282	0.86	22
qft	24	0.8	0.578	1.38	69	0.676	1.18	30
qft	26	3.26	2.08	1.56	78	2.1	1.55	39
qft	27	6.72	4.19	1.60	80	n/a		
qft	28	13.8	8.49	1.62	81	7.77	1.77	44
rand10	20	0.17	0.221	0.77	38	0.373	0.46	11
rand10	22	0.4	0.408	0.98	49	0.611	0.65	16
rand10	24	1.22	1.23	0.98	49	1.54	0.79	20
rand10	26	4.05	5.1	0.80	40	4.79	0.85	21
rand10	27	8.23	6.39	1.29	64	n/a		
rand10	28	21.9	18.4	1.19	60	19.7	1.11	28

on one card against $n + 2$ on four), separate runs of the QFT and the random circuit at $n = 26$ and 27 on one card against $n = 28$ and 29 on four cards (the four card runs at $n = 29$ are not part of Table 12) reach a weak scaling efficiency $T_1(n)/T_4(n + 2)$ of 20 % to 45 %, highest for the QFT and lowest for the random circuit, for the same reason.

Table 13 Slice paths switched off one at a time and together on two and four cards (C), at the largest width of the ablation: end to end time (median) and number of remaps with all paths on and all off.

			T_{e2e} (s)						remaps	
Circuit	C	n	all on	F1 off	F2 off	F3 off	F4 off	all off	on	off
qft	2	28	8.53	9.63	50.5	8.55	8.5	54.1	8	98
qft	4	28	7.92	10	73	8.04	8.04	83.2	9	119
ghz	2	28	2.05	2.03	3.63	5.62	5.7	5.73	1	4
ghz	4	28	1.95	2.04	3.72	7.98	7.85	7.92	1	5
rand10	2	28	18.4	18.2	18.5	18.3	18.2	18.3	16	16
rand10	4	28	20.7	20.5	20.5	20.5	20.5	20.4	20	20
partition heavy	2	28	2.51	2.57	2.57	2.57	2.51	2.45	2	2
partition heavy	4	28	2.64	2.67	2.66	2.66	2.72	2.68	2	2
grover	2	14	2.56	2.73	279	2.27	2.34	151	803	10004
grover	4	14	2.8	2.85	285	2.39	2.08	156	803	10004

Table 14 Baseline design, QFT on four cards with and without the partition diagonal path (the diagonal gates then take the exchange path): routing, execution time, host compute and barrier time (slowest card), PCIe bytes of all cards, and the speedup of the diagonal path (medians).

n	diagonal path	diagonal	exchanges	T_{exec} (s)	host compute (s)	barriers (s)	PCIe (MiB)	speedup
8	on	13	4	0.0356	0.00118	0.0026	0	1.11
	off	0	17	0.0394	0.0021	0.00743	0	
12	on	21	4	0.156	0.00194	0.00345	3	1.07
	off	0	25	0.166	0.00403	0.0146	3	
16	on	29	4	3.58	0.0933	0.077	68	1.13
	off	0	33	4.03	0.338	0.314	68	
18	on	33	4	18	0.136	0.0847	304	1.02
	off	0	37	18.3	0.375	0.269	304	
20	on	37	4	89.9	0.336	0.119	1344	1.01
	off	0	41	91.3	0.954	0.931	1344	

7.6 Ablation of the slice paths and of the compiler rewrite

GHZ preparation and the compiler rewrite F4.

Three GHZ constructions were run on four cards with F4 off and on (Section 4.3). Without F4 the CX chain needs one remap per slice qubit, 5 at every width, and the logarithmic depth construction [32] needs the same number (5): a shorter circuit does not move fewer amplitudes. With F4 the chain needs 1 remap and the remaining CX gates become slice relabelling; at $n = 28$ its end to end time falls from 7.85 s to 1.95 s, a factor of 4.0 (Table 13, F4 off against all on). The chain rooted at the top qubit improves by a factor of 2.3 (2 remaps), while the logarithmic construction keeps 4 remaps, because its CX targets are spread over the register. For this hardware the useful optimisation of GHZ preparation is therefore the placement of the controls, not the depth of the circuit.

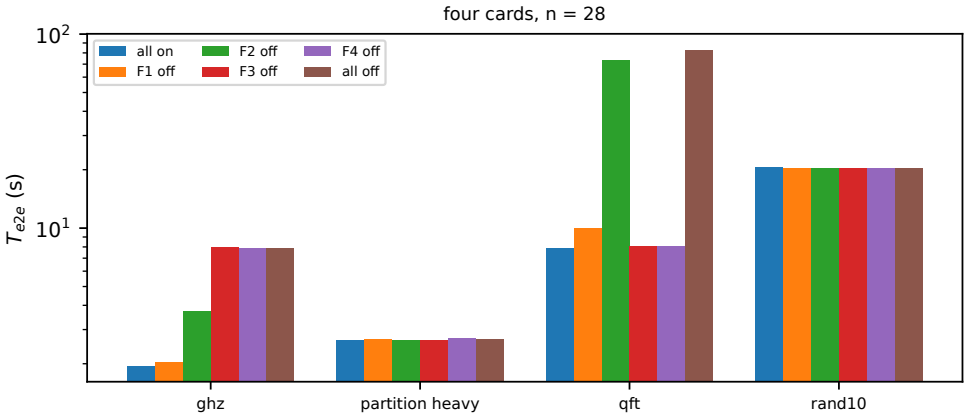


Fig. 5 End to end time of the engine on four cards with the slice paths switched off one at a time and together, at the largest ablation width.

Slice paths.

Table 13 switches the slice paths off one at a time and together; a gate whose path is off falls back to a remap. For the QFT on two cards at $n = 28$ the slice controlled path F2 carries almost the whole benefit: without it the run is 5.9 times slower, with all paths off 6.3 times. Its controlled phase gates on partition qubits have a local target and a slice control, which F2 applies under the slice enable; the slice diagonal path F1, used when the target itself is a slice qubit, adds a factor of 1.13. GHZ preparation relies on slice relabelling and the control promotion (F3, F4) and becomes 2.8 times slower without the slice paths. Grover search at $n = 14$ depends most on F2: its multi controlled gates stay single records only with the slice enable (Section 4.3); without F2 they are decomposed, and the run is 108.8 times slower, 59.0 times with all paths off. The random circuits and the partition heavy circuit are unaffected (1.0 and 1.0): their slice qubits carry non diagonal single qubit gates (the random U3 layers; H and RX in the partition heavy circuit), which need a remap with or without the slice paths, and once a slice qubit is local the following gates on it run locally. For circuits dominated by non diagonal gates on partition qubits the exchange path therefore sets the time. On four cards, where more gates touch slice qubits, the effects are larger: the QFT is 9.2 times slower without F2 and 10.5 times with all paths off, GHZ 4.1 times, and Grover search 101.8 times without F2.

Diagonal path of the baseline design.

In the baseline design a diagonal gate on a partition qubit avoids the exchange between cards but still moves the slice of every card to the host and back, so the PCIe traffic is the same with and without the diagonal path (1344 MiB for the QFT at $n = 20$, Table 14). The path saves host arithmetic (0.95 s against 0.34 s at $n = 20$) and barrier time (0.93 s against 0.12 s), but the run is dominated by the kernel sweeps of the local gates, and the speedup of the diagonal path is only 1.01 to 1.13. Executing diagonal

Table 15 Choice of the partition qubits: the top qubits select the card (default) or qubits 0 and 1 do. End to end time (median), remaps, and the ratio of the two times.

			top qubits		qubits 0 and 1		
Circuit	n	C	T_{e2e} (s)	remaps	T_{e2e} (s)	remaps	ratio
ghz	16	2	0.0258	1	0.0186	3	0.72
ghz	16	4	0.0336	1	0.0581	4	1.73
ghz	20	2	0.0259	1	0.0236	3	0.91
ghz	20	4	0.0349	1	0.042	3	1.21
ghz	24	2	0.158	1	0.109	3	0.69
ghz	24	4	0.15	1	0.16	3	1.07
grover	8	2	0.0389	0	0.0372	1	0.96
grover	8	4	0.0388	0	0.0514	3	1.32
grover	12	2	0.398	201	0.401	201	1.01
grover	12	4	0.567	201	0.817	303	1.44
grover	14	2	2.62	803	3.15	804	1.20
grover	14	4	2.73	803	2.98	804	1.09
qft	16	2	0.0499	5	0.0524	5	1.05
qft	16	4	0.126	10	0.103	8	0.82
qft	20	2	0.11	7	0.105	6	0.95
qft	20	4	0.156	8	0.16	7	1.03
qft	24	2	0.576	8	0.745	6	1.29
qft	24	4	0.655	9	0.649	8	0.99

and controlled gates on partition qubits pays off only when they run on the cards, as in the engine, where switching these paths off makes the QFT 10.5 times slower on four cards.

Assigning the card to qubits 0 and 1 instead of the top qubits changes the end to end time by a factor between 0.69 and 1.73 (Table 15), and no choice is better for every circuit. The QFT acts on all qubits alike and is nearly indifferent. GHZ needs more remaps with the low partition, because its CX chain starts at qubit 0, yet on two cards it runs faster: all its remaps then involve engine qubits inside a card and run as device copies, while with the top partition one remap crosses the cards through host memory. On four cards one remap still crosses the cards, and the low partition is slower. Grover search changes little because its oracle spans the whole register in either case. The top qubit partition is therefore kept as the default, and the runtime accepts any initial placement.

7.7 Comparison with Qiskit on the same host

Qiskit Aer with all hardware threads of the 128 core host is faster than the four card system for almost every circuit and width (Table 16, Fig. 6): the FPGA system is faster in 2 of 37 cases, both GHZ circuits at widths where the multithreaded Aer time is irregular. The gap narrows with the width, because the fixed costs of the FPGA system (compilation, launches, transfers) are amortised and its time is set by HBM passes and exchanges: for the QFT the FPGA system is 8.3 times slower than Aer at $n = 16$ and 1.8 times slower at $n = 30$; for GHZ at $n = 30$ the factor is 1.3. Against a

Table 16 End to end time of the FPGA system (four cards) and simulation time of Qiskit on the host CPU for the same circuits (medians, seconds).

Circuit	n	FPGA T_{e2e}	Statevector	Aer (all threads)	Aer (1 thread)	FPGA/Aer
ghz	12	0.011	0.000908	0.00172	0.00143	6.4
ghz	16	0.0326	0.0255	0.15	0.00762	0.22
ghz	20	0.0447	0.295	0.00459	0.135	9.7
ghz	22	0.0702	1.21	0.213	0.641	0.33
ghz	24	0.181	4.38	0.0867	2.81	2.1
ghz	26	0.555	n/a	0.349	12.2	1.6
ghz	28	2.02	n/a	1.31	52.4	1.5
ghz	30	6.98	n/a	5.23	251	1.3
grover	4	0.00854	0.00302	0.00143	0.00132	6
grover	6	0.0194	0.483	0.00477	0.00426	4.1
grover	8	0.0413	1.93	0.00681	0.00677	6.1
grover	10	0.0864	7.24	0.0226	0.0224	3.8
grover	12	0.55	23.8	0.0708	0.0711	7.8
grover	14	2.73	283	0.376	0.375	7.2
grover	16	22.7	n/a	0.644	1.66	35
qft	12	0.0216	0.00707	0.00572	0.00573	3.8
qft	16	0.12	0.554	0.0144	0.0301	8.3
qft	20	0.157	4.08	0.0296	0.514	5.3
qft	22	0.282	15.9	0.0654	2.4	4.3
qft	24	0.66	62.9	0.211	11.2	3.1
qft	26	2.06	n/a	0.88	50.2	2.3
qft	28	7.63	n/a	3.82	233	2
qft	30	30	n/a	17	1.15e+03	1.8
qft iqft	12	0.0392	0.0148	0.0166	0.0163	2.4
qft iqft	16	0.232	0.528	0.0351	0.0601	6.6
qft iqft	20	0.368	8.25	0.0539	1.07	6.8
qft iqft	22	0.589	33	0.114	4.98	5.2
qft iqft	24	1.8	131	0.392	22.8	4.6
qft iqft	26	6.3	n/a	1.84	104	3.4
qft iqft	28	20.8	n/a	7.21	481	2.9
rand10	12	0.0495	0.0118	0.0109	0.0114	4.5
rand10	16	0.275	0.379	0.0131	0.0401	21
rand10	20	0.371	4.83	0.0286	0.696	13
rand10	22	0.608	18.6	0.066	2.95	9.2
rand10	24	1.66	70.3	0.368	13.7	4.5
rand10	26	4.87	n/a	1.04	57.2	4.7
rand10	28	20.5	n/a	4.38	252	4.7

single CPU thread the FPGA system is faster in 22 of 37 cases and by 38 times for the QFT at $n = 30$. Grover search is the least favourable case: each iteration contains two multi controlled gates over the whole register, which bring different qubits into the cards in turn, so that at $n = 16$ the 201 iterations need 2,417 remaps, the exchanges take 57 % of the end to end time, and the four card system is 35.3 times slower than Aer. We therefore make no claim of speed advantage over a multithreaded CPU simulator on this host; the comparison with GPUs on the same card type is in the companion paper [14].

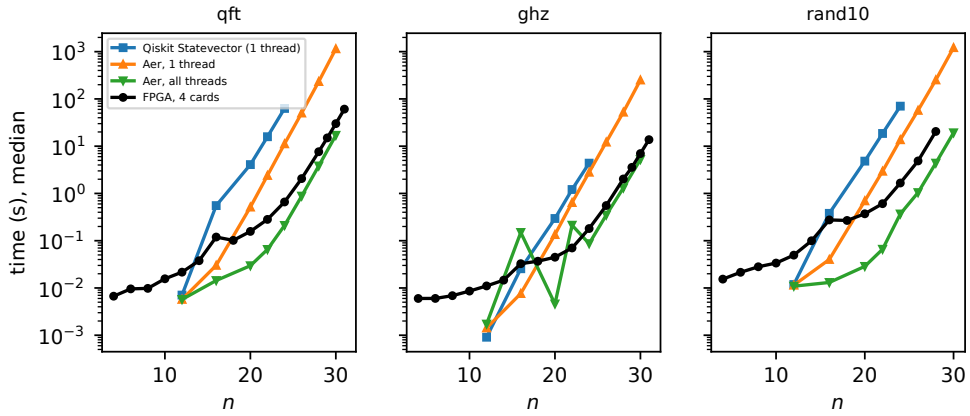


Fig. 6 End to end time of the FPGA system on four cards against Qiskit on the same host (median).

Table 17 End to end time (median, seconds) with the host process and its workers restricted to the first N hardware threads (logical CPUs 0 to $N-1$) of the 128 core, 256 thread host.

				hardware threads				
Design	C	Circuit	n	4	8	16	64	256
baseline	4	qft	16	3.59	3.53	3.54	3.58	3.55
baseline	4	qft	20	90.3	89.1	89.2	89.3	89.2
engine	2	qft	20	0.108	0.108	0.11	0.111	0.109
engine	2	qft	24	0.594	0.598	0.594	0.591	0.568
engine	2	rand10	20	0.213	0.215	0.229	0.223	0.19
engine	2	rand10	24	1.29	1.32	1.29	1.28	1.22
engine	4	qft	20	0.149	0.147	0.151	0.154	0.142
engine	4	qft	24	0.683	0.667	0.67	0.665	0.644
engine	4	rand10	20	0.35	0.354	0.372	0.369	0.353
engine	4	rand10	24	1.6	1.62	1.59	1.6	1.61

7.8 Dependence on the host

Restricting the engine runtime to between 4 and 256 hardware threads changes its end to end time by at most 21 % on two and four cards, without a trend over the number of cores; the largest spreads belong to runs of a few tenths of a second (Table 17). The runtime is one process whose work is compilation, launches and transfers, and the gate arithmetic runs on the cards. The 128 cores of our host are therefore not needed: 4 hardware threads give the same times. A smaller host still needs one PCIe slot per card and host memory for the exchanged half slices; other hosts were not measured. The baseline design varies by 2 % under the same restriction: its QFT time is spent in the kernel sweeps of Section 7.9, and its host arithmetic for the gates on partition qubits is a small part of the run.

Table 18 Baseline design, QFT on four cards: routing counters, measured execution time (median) and the time predicted from microbenchmarks alone (no fitted parameter).

n	sweeps	launches	slice transfers	measured (s)	model (s)	model/measured	sweeps (%)
4	9	2	9	0.0179	n/a	n/a	n/a
8	123	7	17	0.0345	0.0219	0.63	3
12	349	2	25	0.155	0.148	0.96	82
14	504	13	29	0.692	0.74	1.07	94
16	687	15	33	3.5	3.82	1.09	98
18	898	17	37	17.8	19.5	1.09	99
20	1137	19	41	89.2	97.5	1.09	100
22	1404	21	45	440	480	1.09	100
24	1699	23	49	2.13e+03	2.32e+03	1.09	100
26	2022	25	53	1.02e+04	1.11e+04	1.08	100

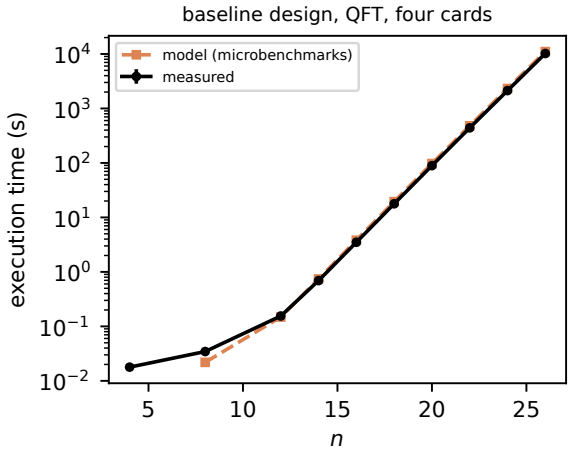


Fig. 7 Baseline design, QFT on four cards: measured execution time and the microbenchmark model.

7.9 A model of the baseline design

The timing curve of the baseline design is nearly flat for small widths and then turns upward. To explain the bend, the model of Table 18 uses only microbenchmarks of the baseline kernel on the same cards (one launch, one gate, collection and distribution of a slice, command round trip and barrier) and the native operation counts of each circuit, obtained by replaying the routing and the lowering of the baseline host (and checked against the counters recorded on the cards); the relative cost of single qubit, CX and CZ operations follows from the loop bounds and initiation intervals of the kernel source. No parameter is fitted to the circuit timings. For $n \geq 12$ the model predicts the measured QFT time to within a factor of 0.96 to 1.09 (Fig. 7). The QFT at $n = 20$ on four cards appears in several campaigns of the baseline design (Tables 18, 14 and 17); the values differ by the variation between campaigns and by whether the

Table 19 Board energy per simulation: time weighted mean of the board power of every used card (XMC sensors, about one reading per second) times the end to end time. The last column is the smallest number of sensor readings inside one run.

Circuit	n	C	T_{e2e} (s)	board power (W)	energy (J)	readings
qft	26	1	3.27	34.3	112	1
qft	26	2	2.16	68.2	147	0
qft	26	4	2.12	136.6	290	0
qft	28	1	13.8	35.0	482	4
qft	28	2	8.49	68.3	580	1
qft	28	4	7.82	137.1	1.07e+03	1
rand10	26	1	4.11	45.3	186	3
rand10	26	2	5.22	76.4	399	3
rand10	26	4	4.53	144.7	655	1
rand10	28	1	21.9	41.1	901	8
rand10	28	2	18.4	77.4	1.42e+03	5
rand10	28	4	20.2	141.5	2.86e+03	3

execution time or the end to end time is quoted, and every comparison in the text takes both of its values from the same campaign.

The bend has a simple origin. The baseline kernel sweeps the whole slice for every native operation, and the time of a sweep doubles with every qubit: 0.35 ms for a slice of 2^{10} amplitudes and 21.56 ms for 2^{16}, i.e. $t_0 = 329$ ns per amplitude, or 85 cycles of the 258 MHz kernel clock. The fixed costs do not grow with the width up to slices of about 2^{14} amplitudes: a kernel launch (0.13 ms), the transfer of a slice to the host and back (0.49 ms at 2^{10} amplitudes), a command round trip between the host processes (359 μs) and a barrier (374 μs); they are set by PCIe, the XRT runtime and the host. With W weighted native operations, N_L launches, N_X slice transfers and N_E exchanges (Table 18), the model reads

$$T(n) \approx W\, t_0\, 2^{n_\ell} + N_L\,(t_L + t_q) + N_X\,(t_X + t_q) + 3N_E\, t_b, \qquad n_\ell = n - \log_2 C, \quad (4)$$

and the bend lies where the first term equals the sum of the others,

$$n^* = \log_2 C + \log_2 \frac{N_L\,(t_L + t_q) + N_X\,(t_X + t_q) + 3N_E\, t_b}{W\, t_0}. \quad (5)$$

With the counts of the QFT for $n = 8$ to 16 on four cards, Eq. (5) gives n^* between 9.9 and 11.0. The measured runs agree: the sweeps take 3 % of the model time at $n = 8$ and more than half from $n = 12$ on, after which the curve follows the exponential growth of the sweeps, multiplied by the number of native operations, which grows with n as well. The bend is therefore set by the ratio of the per amplitude sweep time of the kernel to the fixed costs of PCIe and the host, both measured on the platform; a faster kernel (smaller t_0) moves it to larger n, a faster host path to smaller n.

7.10 Energy

During the simulations the four cards draw together between 137 and 145 W of board power (Table 19). The lowest readings of the same stage, taken while the cards wait parked between runs, add up to 92 W, so about two thirds of the power is static, and the energy of a simulation follows its run time: a QFT at $n = 28$ on four cards takes 1072 J. The card sensors refresh about once per second, so for runs of a few seconds the power is the time weighted mean of the readings in force during the run (zero to three new ones); the table gives the number of readings inside the shortest run. Host power is not included because the CPU energy counters of this host are readable only with administrator rights. Since the power grows with every card and the run time does not fall in proportion (Section 7.5), the energy of a simulation rises with the number of cards: the QFT at $n = 28$ takes 482 J on one card, 580 J on two and 1072 J on four. For widths that fit one card, one card is the most energy efficient configuration of this system.

8 Discussion

What the engine changes.

The baseline design keeps every gate on a partition qubit on the host and sweeps the whole slice once per native operation. Its run time is therefore set by per gate kernel sweeps for wide circuits and by synchronisation for small ones, as the model of Section 7.9 shows without any fitted parameter. The engine removes both costs: gates that need no amplitudes of another slice run on the cards under the slice enable (F1 to F3), consecutive records share one HBM pass, and the state uses all 32 pseudo channels of each card. The QFT at $n = 20$ on four cards takes 89.3 s in the baseline design and 0.157 s in the engine on the same cards, 569 times less.

Where the system stands.

We make no claim of speed advantage. Qiskit Aer with all threads of the host is faster than the four card system for almost every circuit of Table 16; the FPGA system is faster than a single CPU thread in all 14 cases with $n \geq 24$, and its time approaches that of the multithreaded CPU as the width grows. Our measured HBM rate, 95 GB/s per card, is well below the peak of the memory, and the engine is compute bound when a pass carries several operations; a design that issues more amplitude pairs per cycle, or more engines than the eight that routed in complex128, would raise the rate of the cards. The comparison with GPUs on the same card type is in the companion paper [14].

What distribution buys.

Each doubling of the cards adds one qubit of capacity in complex128, from 29 qubits on one card to 31 on four (Section 7.2; verified: 29, 30 and 31 qubits), at the price of exchanges through host memory. Scaling and energy show the price: more cards make most runs slower and every run more energy hungry (Sections 7.5 and 7.10). The slice paths and the compiler rewrite keep the exchanges to a minimum where the

circuit allows it: without them the QFT on four cards is 10.5 times slower and GHZ preparation 4.1 times. Circuits whose slice qubits carry general gates, such as random circuits, gain nothing from them and pay the full exchange cost.

Relation to other multi device simulators.

Distributed CPU and GPU simulators use the same global to local swap and reach far larger widths on many nodes [5–7]. The Trefoil system distributes much larger states over FPGAs with SATA storage, trading bandwidth for capacity [15, 16]. The present design keeps the state in HBM, which limits the width to the memory of the cards but keeps every pass at HBM speed; its weak point is the exchange path between the cards, which peer to peer transfers or direct card to card links would shorten.

Reproducibility.

Every number in this paper is generated by script from the raw measurement files, each of which records the code commit, the bitstream hash and the cards used. The circuits, the transpilation settings, the timing definitions and the measurement protocol are given in Section 6, and the raw data are available as stated in the declarations.

9 Limitations

Thermal limit of one card.

The four cards sit in one host with a shared air path, and the card at PCIe bus 21 runs hottest. With the engine bitstream loaded, that card heats above 84 °C within minutes even when no kernel runs. Every measurement therefore runs under a guard: before each repetition, a card at 85 °C (FPGA) or 80 °C (HBM) is parked on the small bitstream of the baseline design and cooled below 84 °C; a run during which a card reaches 90 °C (FPGA) or 85 °C (HBM) is kept, flagged, and ends that configuration. Table 7 lists the 7 configurations that the guard stopped, all on card 21; where a stopped configuration was run again later, the complete run is the one reported. In one early attempt the cooling wait was done with the engine bitstream still loaded; card 21 reached 96 °C (FPGA) and 91 °C (HBM), its AXI firewall tripped and the driver reset the card. That run is excluded from all results and the procedure was changed as described. The widest runs, at $n = 31$ on four cards, succeeded only after cooling pauses; on this host the time available for long runs at full width is limited by the temperature of one card, not by memory.

Routing limit of the engine count.

A build with sixteen complex128 engines did not route (Section 3). The engine uses eight compute units with four pseudo channels each; it reaches all 32 pseudo channels of a card, but the peak kernel throughput is that of eight engines.

Half of the HBM holds the state.

At full width the state occupies 8 GiB of the 16 GiB of each card (Section 7.2): every pseudo channel holds 2^{24} amplitudes (256 MiB) of a slice next to the engine's

instruction segment, which shares the buffer, and the next power of two would fill the 512 MiB channel completely. Storing the instructions in a separate buffer, on a further HBM port or in on chip memory, would remove this limit and give one more qubit per configuration ($n = 30$ on one card, $n = 32$ on four); we did not build this variant.

Data movement across cards goes through the host.

Gates that need amplitudes of another card are executed after an exchange through host memory over PCIe. Peer to peer transfers between the cards are disabled on this host (the platform reports that they would need 32 GB of PCIe address space per card), and we did not change that system setting. The cost of this path for each circuit family is given in Sections 7.5 and 7.6. This path is why more cards do not make a run faster: on four cards the exchanges take 47 % of the end to end time of the QFT at $n = 28$ and 78 % for the random circuit, and they set the time of every circuit whose slice qubits carry non diagonal gates. Peer to peer transfers or a faster interconnect would shorten exactly this part.

Card count.

The partition assigns the top $\log_2 C$ qubits to the card index, so the number of cards must be a power of two. The runtime accepts any such count, but only $C = 1$, 2 and 4 were run, because the host has four cards. Eight cards would need a host with eight x16 slots, and every further card qubit adds remaps across cards for the circuits that act on it; with the exchange through host memory, the trend of Section 7.5, a growing exchange share with every doubling of the cards, would be expected to continue. We make no measured claim for eight or more cards.

One host, one platform.

All measurements come from one host with four U55C cards. The behaviour of the exchange on hosts with a different PCIe topology or with peer to peer transfers enabled was not measured.

10 Conclusions

We presented a complex128 statevector engine for one, two and four AMD Alveo U55C cards and measured it, together with a baseline design that computes the gates on partition qubits on the host, on four physical cards in one host. The engine executes every gate that needs no amplitudes of another slice on the cards, including diagonal, slice controlled and slice permutation gates on partition qubits and multi controlled gates wider than its tile, packs consecutive gates into HBM passes, and spreads the state over all 32 pseudo channels of every card. It simulates 29 qubits on one card, 30 on two and 31 on four cards, every instance agreeing with an independent reference to double precision rounding (largest amplitude error 2.00×10^{-13}). On the same cards the QFT at $n = 20$ runs 569 times faster than in the baseline design, whose timing curve a microbenchmark model without fitted parameters reproduces to within a factor of 0.96 to 1.09.

The measurements also set the limits. Qiskit Aer on the 128 core host is faster than the four card system for almost all circuits measured. More cards add capacity but not speed, because remaps that involve a card qubit pass through host memory: in 21 of 32 comparisons two or four cards are slower than one, and the energy of a simulation grows with the number of cards. One card of the host limits the widest runs thermally. The next steps are therefore a faster exchange path between the cards, by peer to peer transfers or direct links, and a kernel with a higher rate per card.

Acknowledgments. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India. The research was conducted at the C-DAC Noida Centre under the supervision of Mr. Abhishek Tiwari, Scientist F, Embedded Systems Division, C-DAC Noida. We thank the developers of Qiskit and of the open FPGA tooling on which this study depends.

Declarations

Funding. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India.

Conflict of interest. The author declares no competing interests.

Data availability. The raw measurement records that support the findings of this study (one JSON file per campaign with timestamps, code commit, bitstream hashes and card identifiers), the benchmark circuits in OpenQASM and the scripts that generate every table and figure from them are available from the corresponding author on reasonable request.

Author contribution. N. Ali designed and implemented the HLS kernels, the host runtime and compiler and the benchmark suite, carried out the measurements and wrote the manuscript.

References

[1] Preskill, J.: Quantum computing in the nisq era and beyond. Quantum **2**, 79 (2018) https://doi.org/10.22331/q-2018-08-06-79

[2] Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J.C., Barends, R., Biswas, R., Boixo, S., Brandao, F.G.S.L., Buell, D.A., Burkett, B., Chen, Y., Chen, Z., Chiaro, B., Collins, R., Courtney, W., Dunsworth, A., Farhi, E., Foxen, B., Fowler, A., Gidney, C., Giustina, M., Graff, R., Guerin, K., Habegger, S., Harrigan, M.P., Hartmann, M.J., Ho, A., Hoffmann, M., Huang, T., Humble, T.S., Isakov, S.V., Jeffrey, E., Jiang, Z., Kafri, D., Kechedzhi, K., Kelly, J., Klimov, P.V., Knysh, S., Korotkov, A., Kostritsa, F., Landhuis, D., Lindmark, M., Lucero, E., Lyakh, D., Mandrà, S., McClean, J.R., McEwen, M., Megrant, A., Mi, X., Michielsen, K., Mohseni, M., Mutus, J., Naaman, O., Neeley, M., Neill, C., Niu, M.Y., Ostby, E., Petukhov, A., Platt, J.C., Quintana, C., Rieffel, E.G., Roushan, P., Rubin, N.C., Sank, D., Satzinger, K.J., Smelyanskiy, V., Sung, K.J., Trevithick, M.D., Vainsencher, A., Villalonga, B., White, T., Yao, Z.J., Yeh, P., Zalcman, A., Neven, H., Martinis, J.M.: Quantum supremacy using

a programmable superconducting processor. Nature **574**(7779), 505–510 (2019) https://doi.org/10.1038/s41586-019-1666-5

[3] Smelyanskiy, M., Sawaya, N.P.D., Aspuru-Guzik, A.: qhipster: The quantum high performance software testing environment. arXiv preprint arXiv:1601.07195 (2016) arXiv:1601.07195

[4] Jones, T., Brown, A., Bush, I., Benjamin, S.C.: Quest and high performance simulation of quantum computers. Scientific Reports **9**(1), 10736 (2019) https://doi.org/10.1038/s41598-019-47174-9

[5] Häner, T., Steiger, D.S.: 0.5 petabyte simulation of a 45-qubit quantum circuit. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17), pp. 1–10. ACM, Denver, CO, USA (2017). https://doi.org/10.1145/3126908.3126947

[6] De Raedt, H., Jin, F., Willsch, D., Willsch, M., Yoshioka, N., Ito, N., Yuan, S., Michielsen, K.: Massively parallel quantum computer simulator, eleven years later. Computer Physics Communications **237**, 47–61 (2019) https://doi.org/10.1016/j.cpc.2018.11.005

[7] Imamura, S., Yamazaki, M., Honda, T., Kasagi, A., Tabuchi, A., Nakao, H., Fukumoto, N., Nakashima, K.: mpiqulacs: A distributed quantum computer simulator for a64fx-based cluster systems. arXiv preprint arXiv:2203.16044 (2022) arXiv:2203.16044

[8] Bayraktar, H., Charara, A., Clark, D., Cohen, S., Costa, T., Fang, Y.-L.L., Gao, Y., Guan, J., Gunnels, J., Haidar, A., Hehn, A., Hohnerbach, M., Jones, M., Lubowe, T., Lyakh, D., Morino, S., Springer, P., Stanwyck, S., Terentyev, I., Varadhan, S., Wong, J., Yamaguchi, T.: cuquantum sdk: A high-performance library for accelerating quantum science. In: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 1050–1061 (2023). https://doi.org/10.1109/qce57702.2023.00119

[9] Khalid, A.U., Zilic, Z., Radecka, K.: Fpga emulation of quantum circuits. In: IEEE International Conference on Computer Design: VLSI in Computers and Processors, 2004. ICCD 2004. Proceedings., pp. 310–315 (2004). https://doi.org/10.1109/iccd.2004.1347938

[10] Lee, Y.H., Khalil-Hani, M., Marsono, M.N.: An fpga-based quantum computing emulation framework based on serial-parallel architecture. International Journal of Reconfigurable Computing **2016**, 1–18 (2016) https://doi.org/10.1155/2016/5718124

[11] Pilch, J., Długopolski, J.: An fpga-based real quantum computer emulator. Journal of Computational Electronics **18**(1), 329–342 (2019) https://doi.org/10.1007/s10825-018-1287-5

[12] Mahmud, N., El-Araby, E., Caliga, D.: Scaling reconfigurable emulation of quantum algorithms at high precision and high throughput. Quantum Engineering **1**(2) (2019) https://doi.org/10.1002/que2.19

[13] Liang, S., Lu, Y., Guo, C., Luk, W., Kelly, P.H.J.: Pcq: Parallel compact quantum circuit simulation. In: 2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), pp. 24–31 (2024). https://doi.org/10.1109/fccm60383.2024.00013

[14] Tiwari, A., Ali, N., Khan, N., Mazhar, I., Islur, K., Gupta, H., Kumar, S., Tyagi, P.: FPGA Based Quantum Circuit Simulator: Processing 30 Qubit Systems. Research Square preprint. Under major revision, The Journal of Supercomputing (2026). https://doi.org/10.21203/rs.3.rs-8581344/v1

[15] Niwase, R., Harasawa, H., Yamaguchi, Y., Wei, K., Amano, H., Miyoshi, T.: Enormous-scale quantum state vector calculation with fpga-accelerated sata storages. In: 2023 International Conference on Field Programmable Technology (ICFPT), pp. 288–289 (2023). https://doi.org/10.1109/icfpt59805.2023.00049

[16] Wei, K., Amano, H., Niwase, R., Yamaguchi, Y., Miyoshi, T.: Qu-trefoil: Large-scale quantum circuit simulator working on fpga with sata storages. IEEE Transactions on Computers **74**(4), 1306–1321 (2025) https://doi.org/10.1109/tc.2024.3521546

[17] Guerreschi, G.G., Hogaboam, J., Baruffa, F., Sawaya, N.P.D.: Intel quantum simulator: a cloud-ready high-performance simulator of quantum circuits. Quantum Science and Technology **5**(3), 034007 (2020) https://doi.org/10.1088/2058-9565/ab8505

[18] Zhang, C., Song, Z., Wang, H., Rong, K., Zhai, J.: Hyquas. In: Proceedings of the ACM International Conference on Supercomputing, pp. 443–454 (2021). https://doi.org/10.1145/3447818.3460357

[19] Suzuki, Y., Kawase, Y., Masumura, Y., Hiraga, Y., Nakadai, M., Chen, J., Nakanishi, K.M., Mitarai, K., Imai, R., Tamiya, S., Yamamoto, T., Yan, T., Kawakubo, T., Nakagawa, Y.O., Ibe, Y., Zhang, Y., Yamashita, H., Yoshimura, H., Hayashi, A., Fujii, K.: Qulacs: a fast and versatile quantum circuit simulator for research purpose. Quantum **5**, 559 (2021) https://doi.org/10.22331/q-2021-10-06-559

[20] Markov, I.L., Shi, Y.: Simulating quantum computation by contracting tensor networks. SIAM Journal on Computing **38**(3), 963–981 (2008) https://doi.org/10.1137/050644756

[21] Pednault, E., Gunnels, J.A., Nannicini, G., Horesh, L., Magerlein, T., Solomonik, E., Draeger, E.W., Holland, E.T., Wisnieff, R.: Pareto-efficient quantum circuit simulation using tensor contraction deferral. arXiv preprint arXiv:1710.05867 (2017) arXiv:1710.05867

[22] Chen, Z.-Y., Zhou, Q., Xue, C., Yang, X., Guo, G.-C., Guo, G.-P.: 64-qubit quantum circuit simulation. Science Bulletin **63**(15), 964–971 (2018) https://doi.org/10.1016/j.scib.2018.06.007

[23] Boixo, S., Isakov, S.V., Smelyanskiy, V.N., Babbush, R., Ding, N., Jiang, Z., Bremner, M.J., Martinis, J.M., Neven, H.: Characterizing quantum supremacy in near-term devices. Nature Physics **14**(6), 595–600 (2018) https://doi.org/10.1038/s41567-018-0124-x

[24] Villalonga, B., Boixo, S., Nelson, B., Henze, C., Rieffel, E., Biswas, R., Mandrà, S.: A flexible high-performance simulator for verifying and benchmarking quantum circuits implemented on real hardware. npj Quantum Information **5**(1), 86 (2019) https://doi.org/10.1038/s41534-019-0196-1

[25] Podobas, A.: Q2logic: An coarse-grained architecture targeting schrödinger quantum circuit simulations. arXiv preprint arXiv:2303.01606 (2023) arXiv:2303.01606

[26] Belfore, L.A.: A scalable fpga architecture for quantum computing simulation. arXiv preprint arXiv:2407.06415 (2024) arXiv:2407.06415

[27] Ali, N.: A Native RTL Quantum Statevector Simulator in FP64 on an HBM-Equipped FPGA. Research Square preprint. Under review (2026). https://doi.org/10.21203/rs.3.rs-10115702/v1

[28] Ali, N.: FPGA-Based Real-Time Quantum Error Correction for Shor and Steane Codes. Research Square preprint. Under major revision (2026). https://doi.org/10.21203/rs.3.rs-10115324/v1

[29] Ali, N.: Matrix Product State Engine for FPGA Quantum Circuit Simulation Beyond Five Hundred Qubits. Research Square preprint. Under review (2026). https://doi.org/10.21203/rs.3.rs-10113801/v1

[30] Advanced Micro Devices: Alveo U55C Data Center Accelerator Cards Data Sheet (DS978). Revision 1.3, 2023-06-23. Accessed 2026-10-08 (2023). https://docs.amd.com/r/en-US/ds978-u55c

[31] Advanced Micro Devices: Vitis Unified Software Platform Documentation: Application Acceleration Development (UG1393). Version 2023.2, 2023-12-13. Accessed 2026-10-08 (2023). https://docs.amd.com/r/2023.2-English/ug1393-vitis-application-acceleration

[32] Cruz, D., Fournier, R., Gremion, F., Jeannerot, A., Komagata, K., Tosic, T., Thiesbrummel, J., Chan, C.L., Macris, N., Dupertuis, M., Javerzac-Galy, C.: Efficient quantum algorithms for ghz and w states, and implementation on the ibm quantum computer. Advanced Quantum Technologies **2**(5–6), 1900015 (2019) https://doi.org/10.1002/qute.201900015

[33] Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information, 10th anniversary edn. Cambridge University Press, Cambridge (2010). https://doi.org/10.1017/cbo9780511976667

[34] Boyer, M., Brassard, G., Hoeyer, P., Tapp, A.: Tight bounds on quantum searching. arXiv preprint arXiv:quant-ph/9605034 (1996) https://doi.org/10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P arXiv:quant-ph/9605034

[35] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-eighth Annual ACM Symposium on Theory of Computing - STOC '96, pp. 212–219 (1996). https://doi.org/10.1145/237814.237866

[36] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997) https://doi.org/10.1137/s0097539795293172